Using panic metadata to recover source code information from Rust binaries

2023/12/08

Rust for the Working Reverse Engineer

Rust, Reverse-Engineering

> 🔗 The contents of this article were originally published as a Mastodon thread at `@cxiao@infosec.exchange` .

Introduction

If you've ever looked inside the strings of a Rust binary, you may have noticed that many of these strings are paths to Rust source files (`.rs` extension). These are used when printing diagnostic messages when the program panics, such as the following message:

```
thread 'main' panicked at 'oh no!', src\main.rs:314:5
```

The above message includes both a source file path `src\main.rs` , as well as the exact line and column in the source code where the panic occurred. All of this information is embedded in Rust binaries by default, and is recoverable statically!

Examining these can be useful in separating user from library code, as well as in understanding functionality. This is especially nice because Rust's standard library and the majority of third-party Rust libraries are open-source, so you can use the panic strings to find the relevant location in the source code, and use that to aid in reversing.

Extracting panic metadata

The type that contains this location information is `core::panic::Location`, which has the following definition. It consists of a string slice reference (`&str`), and two unsigned 32-bit integers (`u32`). The string slice reference `&str` represents a view of a string, and is made up of two components: a pointer, and a length.

```
pub struct core::panic::Location<'a> {
    file: &'a str,
    line: u32,
    col: u32,
}
```

Using Binary Ninja, let's look inside a Rust binary at a place where one of the source file path strings is referenced. This is actually a `core::panic::Location` struct, embedded inside the binary.

We can see the following pieces of data, and we can match them against the fields in `core::panic::Location`:

1. A pointer with the value `0x18003adb0`, which is the address where the source file path string resides. This is the pointer component of the string slice `file`.
2. A sequence of bytes with the value `22 00 00 00 00 00 00 00`, which is the little-endian 64-bit integer value `0x22`. This is the length component of the string slice `file`. Note how the length of the path string `library\std\src\sys\windows\mod.rs` is 34 (0x22) bytes (when encoded with UTF-8, which is always the encoding used by `&str`).
3. A sequence of bytes with the value `fc 00 00 00`, which is the little-endian 32-bit integer value `0xfc`. This is the unsigned 32-bit integer `line`.
4. A sequence of bytes with the value `11 00 00 00`, which is the little-endian 32-bit integer value `0x11`. This is the unsigned 32-bit integer `col`.

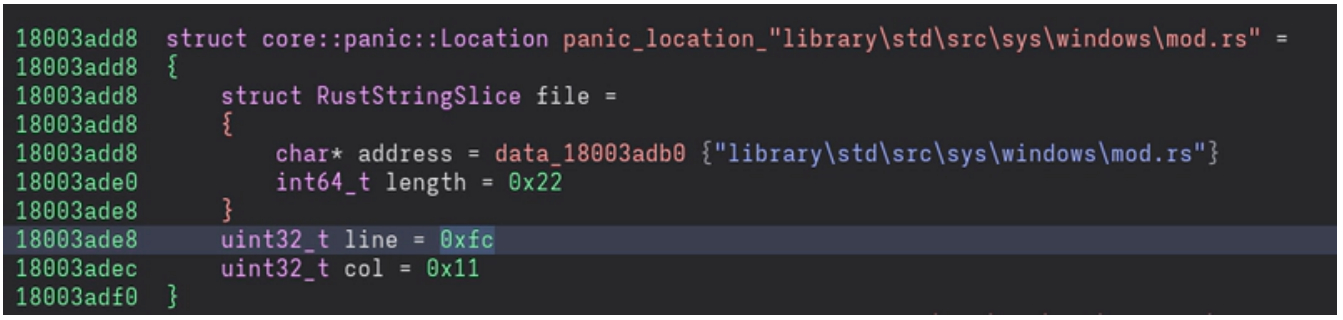
> ⚠️ Caution: Type layouts in compiled Rust binaries are not stable
>
> In this case, the order of the fields in the compiled binary matched against the definition of `core::panic::Location`. However, you cannot rely on this always being the case; the Rust compiler is free to reorder these fields however it wants. Therefore, you must

> do the work of examining the data in your particular binary, and deducing from that what the layout of `core::panic::Location` in your binary is!
>
> For more details on what guarantees the compiler makes (or more importantly, doesn't make) about type layouts, see this section in the Type Layout chapter in The Rust Reference.

Now that we know the layout of `core::panic::Location` in our binary, let's define a new type in Binary Ninja which we can apply to the binary. My type definition for this binary is as follows:

```
struct core::panic::Location
{
    struct RustStringSlice file
    {
        char* address;
        int64_t length;
    };
    uint32_t line;
    uint32_t col;
};
```

The screenshot shows this new type definition applied to the sequence of data, in a nice readable form which shows the line number and column number at a glance.




```
struct core::panic::Location panic_location_"library\std\src\sys\windows\mod.r
{
    struct RustStringSlice file =
    {
        char* address = data_18003adb0 {"library\std\src\sys\windows\mod.rs"}
        int64_t length = 0x22
    }
    uint32_t line = 0xfc
```

```
    uint32_t col = 0x11
}
```

Finding the original library source code

Because the Rust standard library is open-source, we can actually go read the source code at the place that this `core::panic::Location` data points to. The Rust compiler and standard library live in the same Git repository (`rust-lang/rust` on Github) and are released together; the last piece of information we need to find the source code here is the Git commit ID of the Rust compiler / standard library version that was used to create this binary.

This is something we can find by examining some of the other strings in this binary, which contain paths like this, which have the `rustc` commit ID embedded inside them:

```
/rustc/8ede3aae28fe6e4d52b38157d7bfe0d3bceef225\\library\\alloc\\src\\vec\\mod
```

We can now look up the exact location in the source code: `rust-lang/rust` commit `8ede3aae` , File `library\std\src\sys\windows\mod.rs` , Line 252 (0xfc), Column 17 (0x11):

```
fn fill_utf16_buf<F1, F2, T>(mut f1: F1, f2: F2) -> crate::io::Result<T> {
[...]
            if k == n && c::GetLastError() == c::ERROR_INSUFFICIENT_BUFFER {
                n = n.saturating_mul(2).min(c::DWORD::MAX as usize);
            } else if k > n {
                n = k;
            } else if k == n {
                // It is impossible to reach this point.
                // On success, k is the returned string length excluding the n
                // On failure, k is the required buffer length including the n
                // Therefore k never equals n.
                unreachable!(); // ⬅️ This is the location from our binary!
            } else {
[...]
```

This is indeed a place where the code can panic! The `unreachable!()` macro is used to mark locations in the code that should never be reached, in cases where reachability cannot be automatically determined by the compiler. The documentation for `unreachable!()` says:

> This will always `panic!` because `unreachable!` is just a shorthand for `panic!` with a fixed, specific message.

Compare the above snippet of source code with the decompiler's output, at one of the locations where this particular `core::panic::Location` struct is referenced. We can see the following arguments all being passed into the function `sub_18001ee90`, which is the entry point to the panic handling logic (notice how the branch where that function is called is also `noreturn`).

1. The fixed error message for the `unreachable!()` macro (`internal error: entered unreachable code`)
2. The length of that error message string (0x28 characters)
3. The address of that `core::panic::Location` struct.

```
} else {
    if (GetLastError() != ERROR_INSUFFICIENT_BUFFER) {
        sub_18001ee90("internal error: entered unreachable code", 0x28, &panic
        noreturn
    }
    uint64_t n_6 = n * 2
    if (n_6 u>= 0xffffffff) {
        n_6 = 0xffffffff
    }
    n_2 = n_6
[...]
```

The information passed in the arguments is used to construct the message that the program emits when it panics, which in this case will look something like the following:

```
thread 'main' panicked at 'internal error: entered unreachable code', library\
```

We can also see several other features which appear in the original source code at this location:

1. The check of the `GetLastError()` result against the `ERROR_INSUFFICIENT_BUFFER` error code.
2. The saturating multiplication of the variable `n`.

Rust binaries without panic metadata

Note that this location information is only embedded into Rust binaries if the developer uses the default panic behaviour, which unwinds the stack, cleans up memory, and collects information to show in the panic message and in backtraces. You can read more about the details of the default panic implementation in the Rust standard library in the Rust Compiler Dev book.

~~Developers can trivially strip the location information by just putting `panic = 'abort'` when specifying the build profile in their `Cargo.toml` build configuration file; this will cause the program to immediately abort on panic instead, without taking any further actions.~~
(2024-09-06 Edit: This is not correct! This changes the process behaviour upon panic and removes the code that handles unwinds, but does not remove the location metadata. A corrected explanation for how to remove the location metadata is below. Thank you to @mhnap for pointing this out in the comments.)

One way for developers to strip the location information is using the Rust compiler (rustc)'s unstable `location-detail` feature. The details of the `location-detail` feature are documented in the Rust Unstable Book. You can remove all filename, line number, and column number information, or remove only a subset of those three.

You will need the nightly version of the Rust toolchain for this feature. You can install it via:

```
rustup toolchain install nightly
```

Once you have the nightly version of the toolchain, you can run Cargo and tell it to run the nightly toolchain (`+nightly`), and pass the unstable `location-detail` flag to rustc (`RUSTFLAGS="-Zlocation-detail=none"`):

```
RUSTFLAGS="-Zlocation-detail=none" cargo +nightly build --release
```

Here is an example program that I built this way:

```
fn main() {
    panic!("This program panics immediately on running");
}
```

And here is the output of this program. The `<redacted>:0:0:` there is the literal output.

```
thread 'main' panicked at <redacted>:0:0:
This program panics immediately on running
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

You will notice that if you run `strings` on this program, or open it in any RE tool, that there is still some filename, line number, and column number information remaining in the binary. This is panic location metadata from the Rust standard library. It is present because, by default, the standard library that the Rust toolchain uses is a precompiled version, with all of the panic location metadata still intact.

```
$ strings target/release/rust-reversing-panic-information-scratch | grep "\.rs

entity not foundconnection resethost unreachableno storage spaceinvalid filena
!/rustc/9c01301c52df5d2d7b6fe337707a74e011d68d6f/library/alloc/src/collections
/rustc/9c01301c52df5d2d7b6fe337707a74e011d68d6f/library/core/src/str/pattern.r
/rustc/9c01301c52df5d2d7b6fe337707a74e011d68d6f/library/core/src/slice/sort/st
/rustc/9c01301c52df5d2d7b6fe337707a74e011d68d6f/library/alloc/src/vec/mod.rs/r
/rust/deps/addr2line-0.22.0/src/lib.rs/rust/deps/addr2line-0.22.0/src/functior
cannot access a Thread Local Storage value during or after destructionstd/src/
use of std::thread::current() is not possible after the thread's local data ha
std/src/io/stdio.rs
std/src/io/mod.rs
std/src/path.rs
.std/src/sync/once.rs
std/src/../../backtrace/src/symbolize/mod.rs -
[...]
```

If you open the binary in an RE tool, you will notice that the location structures for non-standard-library panic location information is still present, but filenames have been replaced with the string `<redacted>` and line/column numbers have been replaced with 0. The panic location information for the standard library is still intact, however. Here is part of my binary, annotated in Binary Ninja, with the `core::panic::Location` structure from this blog post applied:

```
100044248  struct core::panic::Location data_100044248 =
100044248  {
```

```
100044248      struct RustStringSlice file =
100044248      {
100044248          char* address = data_1000371ba {"<redacted>"}
100044250          int64_t length = 0xa
100044258      }
100044258      uint32_t line = 0x0
10004425c      uint32_t col = 0x0
100044260  }
100044260  struct core::panic::Location data_100044260 =
100044260  {
100044260      struct RustStringSlice file =
100044260      {
100044260          char* address = data_1000377fd {"/rustc/9c01301c52df5d2d7b6
100044268          int64_t length = 0x4f
100044270      }
100044270      uint32_t line = 0x5c8
100044274      uint32_t col = 0x14
100044278  }
```

If you would like to totally strip this information, you can compile the standard library yourself, and apply the `-Z location-detail=none` flag to it to also remove this location information from the standard library. You can do this with the unstable `build-std` feature in Cargo, documented here in the Cargo book.

As it says in the documentation there, there are a few things you need to do for this:

1. Obtain a copy of the Rust standard library source code, via running

```
rustup component add rust-src --toolchain nightly
```

2. Build the binary, passing the unstable `-Z build-std` flag to Cargo. Note that this flag is passed to *Cargo*, instead of `-Zlocation-detail=none` from above which was passed to rustc. The entire invocation looks like this (this feature also requires that you explicitly pass a `--target` flag):

```
$ RUSTFLAGS="-Zlocation-detail=none" cargo +nightly build -Z build-std --targe
```

You should now find that there are no source file names at all inside the binary.

```
$ strings target/aarch64-apple-darwin/release/rust-reversing-panic-informatior
```

Here is this binary again annotated in Binary Ninja, showing that the `<redacted>:0:0:` file / line / column information is now used for all panic structures, including those in the standard library.

```
100048370  struct core::panic::Location data_100048370 =
100048370  {
100048370      struct RustStringSlice file =
100048370      {
100048370          char* address = 0x10003a470 {"<redacted>"}
100048378          int64_t length = 0xa
100048380      }
100048380      uint32_t line = 0x0
100048384      uint32_t col = 0x0
100048388  }
100048388  struct core::panic::Location data_100048388 =
100048388  {
100048388      struct RustStringSlice file =
100048388      {
100048388          char* address = data_10003a54c {"<redacted>"}
100048390          int64_t length = 0xa
100048398      }
100048398      uint32_t line = 0x0
10004839c      uint32_t col = 0x0
1000483a0  }
```

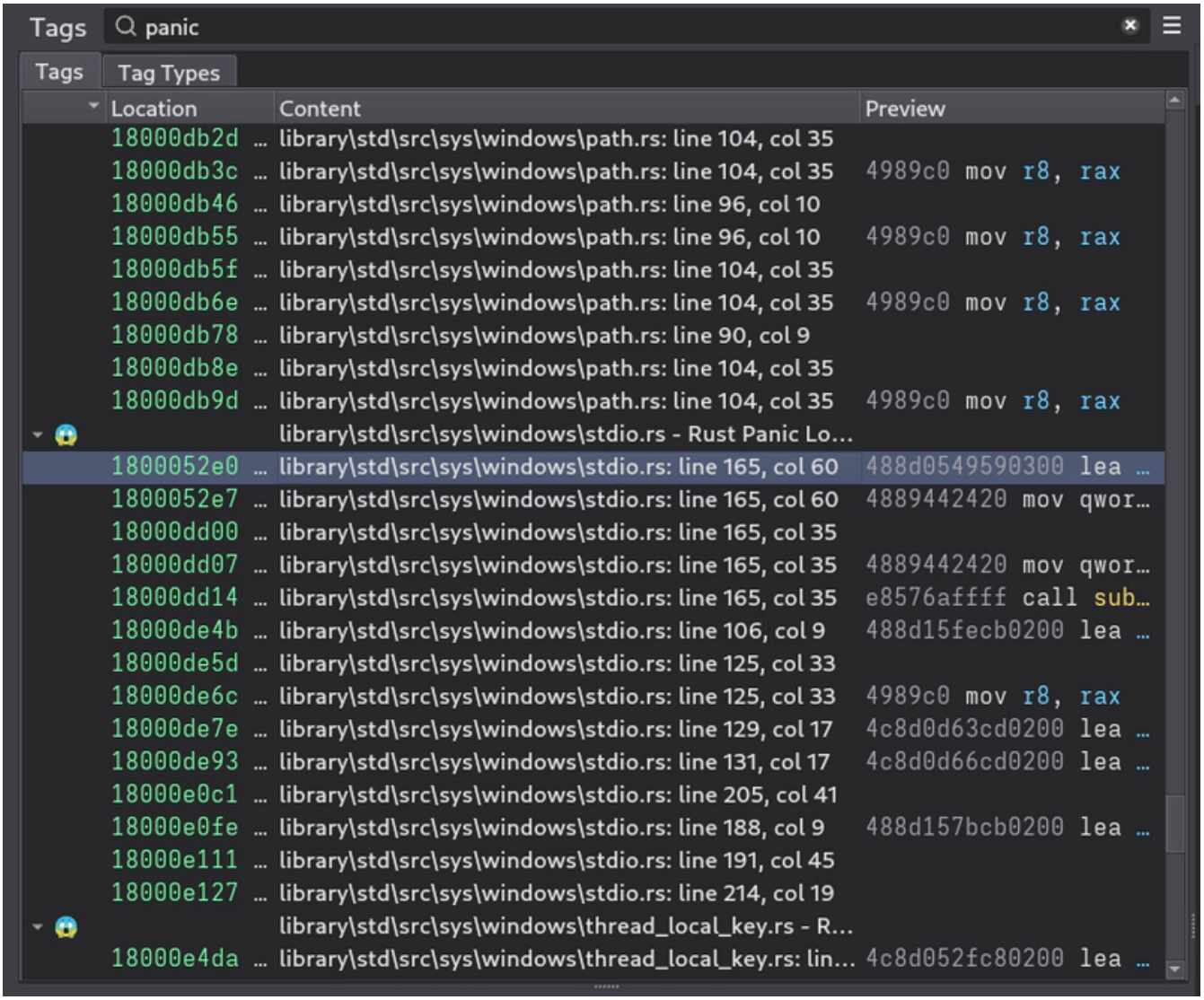
To learn more about various methods of stripping metadata from Rust binaries, the min-sized-rust repository is a great reference for various compiler and toolchain features, and their effects on the binary.

Developers can also provide their own custom panic handlers - you can learn more about this in my previous Mastodon post about the panic handlers used by the Rust code in the Windows kernel.

Of course, a lot of Rust malware out there doesn't even bother taking the more basic step of stripping symbols, much less this panic information...

Next steps

You can take this a step further and write a script that automatically extracts all of this panic location metadata, in your reverse engineering tool of choice. I wrote a quick Binary Ninja script that goes and extracts all panic location metadata, finds the locations in the code where they are referenced, and then annotates each location with a tag that displays the extracted source file information. Here are the tags generated by my script, run on a piece of Rust malware targeting macOS systems, *RustBucket*.



That's all! I will release the metadata extraction script for this at some point.

1 comment · 3 replies *– powered by giscus*

Oldest Newest

Write Preview Aa

Sign in to comment

Sign in with GitHub

mhnap Aug 26, 2024

> Developers can trivially strip the location information by just putting panic = 'abort' when specifying the build profile in their Cargo.toml build configuration file;

Is this still true? I have set `panic = "abort"` but still I can view the panic location information in the error log and binary.

↑ 1 3 replies

cxiao Sep 7, 2024 Owner edited

Hey **@mhnap**, thank you for your comment, and my apologies for taking so long to reply!

I think I was just completely wrong when I said that `panic = "abort"` was the way to remove the location information. This changes the process behaviour upon panic and removes the code that handles unwinds, but does not remove the location metadata. I will correct this blog post in a bit.

The actual option to remove this information this is the Rust compiler (rustc)'s unstable `location-detail` feature. The details of this feature are documented in the Rust Unstable Book. You can remove all filename, line number, and column number information, or remove only a subset of those three.

You will need the nightly version of the Rust toolchain for this feature. You can install it via:

```
rustup toolchain install nightly
```

Once you have the nightly version of the toolchain, you can run Cargo and tell it to run the nightly toolchain (`+nightly`), and pass the unstable `location-detail` flag to rustc (`RUSTFLAGS="-Zlocation-detail=none"`):

```
RUSTFLAGS="-Zlocation-detail=none" cargo +nightly build --release
```

Here is an example program that I built this way:

```
fn main() {
    panic!("This program panics immediately on running");
}
```

And here is the output of this program. The `<redacted>:0:0:` there is the literal output.

```
thread 'main' panicked at <redacted>:0:0:
This program panics immediately on running
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

You will notice that if you run `strings` on this program, or open it in any RE tool, that there is still some filename, line number, and column number information remaining in the binary. This is panic location metadata from the Rust standard library. It is present because, by default, the standard library that the Rust toolchain uses is a precompiled version, with all of the panic location metadata still intact.

```
$ strings target/release/rust-reversing-panic-information-scratch | grep "\.rs"

entity not foundconnection resethost unreachableno storage spaceinvalid filenames
!/rustc/9c01301c52df5d2d7b6fe337707a74e011d68d6f/library/alloc/src/collections/bt
/rustc/9c01301c52df5d2d7b6fe337707a74e011d68d6f/library/core/src/str/pattern.rsre
/rustc/9c01301c52df5d2d7b6fe337707a74e011d68d6f/library/core/src/slice/sort/stabl
/rustc/9c01301c52df5d2d7b6fe337707a74e011d68d6f/library/alloc/src/vec/mod.rs/rust
/rust/deps/addr2line-0.22.0/src/lib.rs/rust/deps/addr2line-0.22.0/src/function.rs
cannot access a Thread Local Storage value during or after destructionstd/src/thr
use of std::thread::current() is not possible after the thread's local data has b
std/src/io/stdio.rs
std/src/io/mod.rs
std/src/path.rs
.std/src/sync/once.rs
std/src/../../backtrace/src/symbolize/mod.rs -
[...]
```

If you open the binary in an RE tool, you will notice that the location structures for non-standard-library panic location information is still present, but filenames have been replaced with the string `<redacted>` and line/column numbers have been replaced with 0. The panic location information for the standard library is still intact, however. Here is part of my binary, annotated in Binary Ninja, with the `core::panic::Location` structure from this blog post applied:

```
100044248  struct core::panic::Location data_100044248 =
100044248  {
100044248      struct RustStringSlice file =
100044248      {
100044248          char* address = data_1000371ba {"<redacted>"}
100044250          int64_t length = 0xa
100044258      }
100044258      uint32_t line = 0x0
10004425c      uint32_t col = 0x0
```

```
100044260  }
100044260  struct core::panic::Location data_100044260 =
100044260  {
100044260      struct RustStringSlice file =
100044260      {
100044260          char* address = data_1000377fd {"/rustc/9c01301c52df5d2d7b6fe3
100044268          int64_t length = 0x4f
100044270      }
100044270      uint32_t line = 0x5c8
100044274      uint32_t col = 0x14
100044278  }
```

If you would like to totally strip this information, you can compile the standard library yourself, and use the `-Z location-detail=none` flag for the compilation, to also remove this location information from the standard library. You can do this with the unstable `build-std` feature in Cargo (https://doc.rust-lang.org/cargo/reference/unstable.html#build-std).

As it says in the documentation there, there are a few things you need to do for this:

1. Obtain a copy of the Rust standard library source code, via running

```
rustup component add rust-src --toolchain nightly
```

2. Build the binary, passing the unstable `-Z build-std` flag to Cargo. Note that this flag is passed to *Cargo*, instead of `-Zlocation-detail=none` from above which was passed to rustc. The entire invocation looks like this (this feature also requires that you explicitly pass a `--target` flag):

```
$ RUSTFLAGS="-Zlocation-detail=none" cargo +nightly build -Z build-std --target a
```

You should now find that there are no source file names at all inside the binary.

```
$ strings target/aarch64-apple-darwin/release/rust-reversing-panic-information-sc
```

Here is this binary again annotated in Binary Ninja, showing that the `<redacted>:0:0:` file / line / column information is now used for all panic structures, including those in the standard library.

```
100048370  struct core::panic::Location data_100048370 =
100048370  {
100048370      struct RustStringSlice file =
100048370      {
100048370          char* address = 0x10003a470 {"<redacted>"}
100048378          int64_t length = 0xa
100048380      }
100048380      uint32_t line = 0x0
100048384      uint32_t col = 0x0
100048388  }
100048388  struct core::panic::Location data_100048388 =
100048388  {
100048388      struct RustStringSlice file =
```

```
100048388      {
100048388          char* address = data_10003a54c {"<redacted>"}
100048390          int64_t length = 0xa
100048398      }
100048398      uint32_t line = 0x0
10004839c      uint32_t col = 0x0
1000483a0  }
```

cxiao Sep 7, 2024 Owner

I have now updated the blog post with my explanation above. Thanks again for pointing this out, and let me know if you have any other questions or comments!